Course Project Report
Group 10

Submission date:

We declare that:

(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;

(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration; (iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;

(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
XIE Yufeng	13974667
CHENG Haoxuan	14128307
Lam Hung Ming	14276107

Binary tree:
https://github.com/yuf44643ai/inventory_system/tree/main/binary_tree

Definition: A binary tree is a hierarchical data structure where each node has at most two children, called the left child and the right child.

Structure: root (top node) and leaf (nodes with no children)

Each node contains: data + left pointer + right pointer

Key Terms: parent node (the node directly above a given node), child node (the node directly below a given node), subtree (a given node treated as a root), path (a sequence of nodes connected by edges), height (edges from root to deepest leaf).

Types:

1. **Full Binary Tree:** 0 or 2 children per node
2. **Complete Binary Tree:** All levels filled except last, which is left-filled
3. **Perfect Binary Tree:** All internal nodes have 2 children, all leaves same level
4. **Skewed Binary Tree:** All nodes have only one child (left or right)

Traversal method:

- **Depth-First Traversal:**
  1. Inorder (Left–Root–Right): gives sorted order for Binary Search Trees (BST)
  2. Preorder (Root–Left–Right): used to copy tree
  3. Postorder (Left–Right–Root): used to delete tree
- **Breadth-First Traversal:**
  1. Level-order: visits level by level

Common Trees & Uses:

1. BST: fast search/insert/delete (O(log n) average)
2. Decision tree: machine learning
3. Expression tree: compilers

4. Huffman tree: data compression

Applications:

Database indexing, file systems, Expression parsing, priority queues (heaps), AI (game trees), graphics (BSP trees), cryptography (Merkle trees)

Shell sort: https://github.com/yuf44643-ai/inventory_system/tree/main/shell_sort

Definition: It is an in-place comparison sorting algorithm and an optimized version of the insertion sort.

How it works:

1. Choose a Gap Sequence: Shell sort starts by picking a gap (e.g., n/2, then n/4, …).
2. Sort Sublists: For a given gap, the array will be separated to several sublists. Each sublist is sorted independently using insertion sort.
3. Reduce the Gap: After one pass, the gap is reduced (e.g., gap = gap/2).
4. Final Pass: When the gap becomes 1, the whole array is sorted with one ordinary insertion sort. As the array is nearly ordered, the final pass is fast.

Characteristics:

1. Time Complexity:

The time complexity of the Shell Sort algorithm is highly dependent on the chosen gap sequence and is not a single, fixed value.

Case	Time complexity	Explanation
Best	O(n log n)	when the array is already nearly sorted (depends on gap sequence).
Average	O(n**2)	depending on the chosen gap sequence.
Worst	O(n**2)	with simple gap sequences like n/2, n/4, …, 1.

2. **Space Complexity:** It is an in-place sorting algorithm.
3. **Stability:** It is not a stable sort(equal elements may not retain their original relative order).
4. **Adaptability:** It performs well in embedded systems and real-time applications.

Followed Key Details:

1. **Dependency on Gap Sequence:**Common gap sequences (e.g., Knuth, Hibbard, Sedgewick) are designed to improve performance over the basic O(n**2) worst case.
2. **Optimization of Insertion Sort:**By moving elements over large distances initially, Shell Sort reduces the number of swaps needed in the final, small-gap passes, significantly improving practical performance.
3. **Development of Shell sort:**The development of Shell sort involves only changing the gap sequence under the original framework. The famous Knuth sequence changes the gap to (3**k – 1)/2 in 1973, whose worst-case time complexity is about O(n**1.5). The latest sequence improves the worst-case time complexity to better than O(n**1.25).
4. **Application of Shell sort:** It is used in embedded systems and hardware where memory is limited and a simple, in-place sort is needed, for sorting medium-sized datasets, or as a subroutine in hybrid sorts.

Reference:

Python binary tree https://www.w3schools.com/python/python_dsa_binarytrees.asp

Appendix

The source images for the introduction to binary trees and shell sort are in the Github repository.